```
rooms = {}

rooms["A"] = input("Enter status of Room A (Dirty/Clean): ").capitalize()
rooms["B"] = input("Enter status of Room B (Dirty/Clean): ").capitalize()

location = input("Enter starting location (A/B): ").upper()

while True:
    if rooms[location] == "Dirty":
        print("\nLocation:", location)
        print("Percept:", location, "is Dirty")
        print("Action: SUCK")
        rooms[location] = "Clean"

    else:
        if location == "A":
            print("\nLocation:", location)
            print("Percept:", location, "is Clean")
            print("Action: Move Right")
            location = "B"
        else:
            print("\nLocation:", location)
            print("Percept:", location, "is Clean")
            print("Action: Move Left")
            location = "A"

    if rooms["A"] == "Clean" and rooms["B"] == "Clean":
        print("\nBoth rooms are clean.")
        print("Agent Goal Achieved.")
        break
```

5

```
from collections import deque

def bfs_path(grid, start, goal):
    rows, cols = len(grid), len(grid[0])

    queue = deque([start])
    parent = {start: None}

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        r, c = queue.popleft()

        if (r, c) == goal:
```

```
            path = []
            current = (r, c)

            while current is not None:
                path.append(current)
                current = parent[current]

            return path[::-1]

        for dr, dc in moves:
            nr, nc = r + dr, c + dc

            if (0 <= nr < rows and 0 <= nc < cols
                and grid[nr][nc] == 0
                and (nr, nc) not in parent):

                parent[(nr, nc)] = (r, c)
                queue.append((nr, nc))

    return None


# Take input from the user
rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

grid = []

print("Enter grid values (0 = Free, 1 = Obstacle):")

for i in range(rows):
    row = list(map(int, input(f"Enter row {i}: ").split()))

    while len(row) != cols or any(x not in (0, 1) for x in row):
        print("Invalid input! Enter exactly", cols, "values (0 or 1).")
        row = list(map(int, input(f"Enter row {i}: ").split()))

    grid.append(row)

print("\nEnter starting position:")
start_r = int(input("Start row: "))
start_c = int(input("Start column: "))
start = (start_r, start_c)

print("\nEnter goal position:")
goal_r = int(input("Goal row: "))
goal_c = int(input("Goal column: "))
goal = (goal_r, goal_c)
```

```
# Validate start and goal
if not (0 <= start_r < rows and 0 <= start_c < cols
        and 0 <= goal_r < rows and 0 <= goal_c < cols):
    print("Invalid start or goal position!")

elif grid[start_r][start_c] == 1 or grid[goal_r][goal_c] == 1:
    print("Start and goal must be free cells (0)!")

else:
    path = bfs_path(grid, start, goal)

    if path:
        print("\nPath found:", path)
        print("Steps:", len(path) - 1)
    else:
        print("No path exists")
```

6

```
# Maze Solving using DFS
# 0 = Open Path
# 1 = Wall

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

maze = []

print("Enter maze row by row (0 = Open, 1 = Wall):")

for i in range(rows):
    row = list(map(int, input().split()))
    maze.append(row)

start_x = int(input("Enter starting row: "))
start_y =int(input("Enter starting column: "))
goal_x = int(input("Enter goal row: "))
goal_y = int(input("Enter goal column: "))

visited = set()

def dfs(x, y):
    if x < 0 or y < 0 or x >= rows or y >= cols:
        return False

    if maze[x][y] == 1:
        return False
```

```
    if (x, y) in visited:
        return False

    print("Visited:", (x, y))

    if (x, y) == (goal_x, goal_y):
        print("\nGoal Reached!")
        return True

    visited.add((x, y))

    if dfs(x + 1, y):
        return True

    if dfs(x - 1, y):
        return True

    if dfs(x, y + 1):
        return True

    if dfs(x, y - 1):
        return True

    return False


if (0 <= start_x < rows and
    0 <= start_y < cols and
    maze[start_x][start_y] == 0 and
    0 <= goal_x < rows and
    0 <= goal_y < cols and
    maze[goal_x][goal_y] == 0):

    print("\nStarting DFS...\n")

    if not dfs(start_x, start_y):
        print("\nNo path found!")

else:
    print("Invalid start or goal position!")
```

```python
def solve_n_queens(n):
    board = [-1] * n
    solutions = []

    def is_safe(row, col):
        for r in range(row):
            c = board[r]
            if c == col or abs(c - col) == abs(r - row):
                return False
        return True

    def backtrack(row):
        if row == n:
            solutions.append(board[:])
            return

        for col in range(n):
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1

    backtrack(0)
    return solutions


def print_board(solution):
    n = len(solution)
    for row in range(n):
        line = ""
        for col in range(n):
            line += "Q " if solution[row] == col else ". "
        print(line)
    print()


n = int(input("Enter the value of N: "))

if n < 1:
    print("Please enter a positive integer.")
else:
    solutions = solve_n_queens(n)

    print("Total solutions found:", len(solutions))

    if len(solutions) > 0:
        print("\nFirst solution:")
        print_board(solutions[0])
    else:
```

```
        print("No solution exists.")
```

8

```
import re

rules = [
    (r'.*\b(hi|hello|hey)\b.*', ["Hello! How can I help you today?", "Hi
there!"]),
    (r'.*\bhow are you\b.*', ["I'm just a program, but I'm doing well!"]),
    (r'.*\bmy name is (.*)', ["Nice to meet you, {0}!"]),
    (r'.*\b(bye|goodbye)\b.*', ["Goodbye! Have a great day!"]),
    (r'.*\bwhat is your name\b.*', ["I am a simple rule-based chatbot."]),
    (r'.*\bhelp\b.*', ["I can chat about simple things. Try greeting me!"]),
    (r'.*', ["Sorry, I didn't understand that. Can you rephrase?"]),  #
default rule
]

def chatbot_response(user_input):
    user_input = user_input.lower().strip()

    for pattern, responses in rules:
        match = re.match(pattern, user_input)

        if match:
            response = responses[0]

            if match.groups():
                response = response.format(*match.groups())

            return response

    return "I don't understand."


print("Chatbot: Hello! Type 'bye' to exit.")

while True:
    user_input = input("You: ")

    if re.match(r'.*\b(bye|goodbye)\b.*', user_input.lower()):
        print("Chatbot:", chatbot_response(user_input))
        break

    print("Chatbot:", chatbot_response(user_input))
```